

Pipeline OpenTelemetry End-to-End con Jaeger y Prometheus en GCP y AWS

Reporte técnico — Laboratorio de Observabilidad
Ilich — Universidad de La Sabana — Agosto de 2026

Resumen ejecutivo

Este laboratorio implementa un pipeline de observabilidad completo basado en OpenTelemetry (OTel) sobre una arquitectura de dos microservicios con dependencia HTTP y acceso a base de datos: **service-a** (órdenes, FastAPI) invoca por HTTP a **service-b** (inventario, FastAPI + PostgreSQL). Ambos servicios emiten los tres pilares de la observabilidad — trazas distribuidas, métricas y logs estructurados — a través del SDK de OpenTelemetry para Python, hacia un OTel Collector que los enruta a Jaeger (trazas), Prometheus (métricas) y Loki/Cloud Logging/CloudWatch (logs).

La correlación cross-signal se logra mediante la propagación de contexto **W3C TraceContext**: el mismo `trace_id` aparece en los spans de Jaeger, en cada línea de log JSON y permite acotar las métricas del mismo request, verificado experimentalmente con la traza `26c5794236a8917f9830aef5281c4725` que atraviesa ambos servicios y la base de datos. El despliegue se define como infraestructura como código para dos nubes: GKE en GCP (Terraform + Helm) y ECS Fargate en AWS (Terraform, con el Collector ADOT como sidecar). Finalmente, un benchmark con k6 (75 usuarios concurrentes, 5 minutos por escenario, más de 128.000 peticiones en total) cuantifica el overhead de la instrumentación en un entorno restringido de 2 vCPU: +371,9 ms en p99, +42,3 puntos porcentuales de CPU y +114 MB de RAM, resultados que se analizan y contextualizan en la sección 6.

1. Arquitectura de la solución

La arquitectura sigue el patrón recomendado por OpenTelemetry de separar la *emisión* (SDK en el proceso de la aplicación), el *procesamiento* (Collector como pieza intermedia) y el *almacenamiento/consulta* (backends especializados por señal). Las aplicaciones exportan por OTLP gRPC al Collector; ningún servicio conoce los backends finales, lo que permite cambiar de Jaeger a X-Ray o de Prometheus a AMP sin tocar código.

```
k6 (75 VUs) → service-a (FastAPI, órdenes) → [HTTP + traceparent W3C] → service-b  
(FastAPI, inventario) → PostgreSQL  
service-a, service-b → [OTLP gRPC 4317: trazas + métricas + logs] → OTel Collector  
Collector → trazas → Jaeger | métricas → Prometheus (:8889) | logs → Loki (OTLP) – y  
Grafana como capa de visualización unificada
```

En GCP el Collector se despliega como Deployment en GKE Autopilot junto con Jaeger, y los logs se envían a Cloud Logging mediante el exporter `googlecloud` con Workload Identity (sin claves). En AWS el Collector (imagen ADOT) corre como sidecar de cada task de ECS Fargate: las trazas van a X-Ray, las métricas a Amazon Managed Prometheus vía remote write con SigV4 y los logs a CloudWatch Logs.

2. Fase 1 — Instrumentación con el SDK de OpenTelemetry

2.1 Decisiones de diseño

Se eligió **Python + FastAPI** por la madurez de sus librerías de auto-instrumentación (`opentelemetry-instrumentation-fastapi/httpx/sqlalchemy`) y la legibilidad del código. La inicialización de telemetría está centralizada en un módulo `telemetry.py` por servicio, que configura los tres proveedores (`TracerProvider`, `MeterProvider`, `LoggerProvider`) con un `Resource` común (`service.name`, `service.version`, `deployment.environment`) y exporta todo por OTLP gRPC. Una variable de entorno `OTEL_ENABLED` permite desactivar por completo la instrumentación, lo que habilita la línea base limpia del benchmark de la Fase 4.

2.2 Auto-instrumentación y spans custom

La auto-instrumentación cubre el servidor HTTP de ambos servicios (FastAPI), el cliente HTTP de service-a (HTTPX, que inyecta el header `traceparent`) y el acceso a datos de service-b (SQLAlchemy, que genera spans `connect` y `SELECT inventory`). Sobre esa base se instrumentó manualmente la lógica de negocio crítica con spans custom enriquecidos con atributos de dominio:

Servicio	Span custom	Atributos de negocio
service-a	orders.calculate_total	app.order.subtotal, app.order.discount_applied, app.order.total
service-b	inventory.check_stock	app.item.sku, app.item.stock, app.item.in_stock

2.3 Los tres pilares emitidos

Pilar	Qué se emite	Ruta
Trazas	Spans auto (HTTP/DB) + 2 custom; 11 spans por request típico	SDK → OTLP gRPC → Collector → Jaeger / X-Ray
Métricas	http.server.duration (histograma), orders_processed_total, inventory_stock_checks_total, inventory_check_duration_ms, order_total_value	SDK → OTLP → Collector → Prometheus :8889 / AMP
Logs	JSON estructurado: timestamp, severity, service, message, trace_id, span_id (inyectados por <code>LoggingInstrumentor</code>)	stdout + SDK → OTLP → Collector → Loki / Cloud Logging / CloudWatch

Ejemplo real de log emitido durante el laboratorio, donde se aprecia el `trace_id` compartido con la traza:

```
{"timestamp": "2026-08-16T03:19:08.198769+00:00", "severity": "INFO", "service": "service-a", "message": "Orden ORD-647893 procesada: SKU-002 x9 total=273.11", "trace_id": "34388a8870ac9c1048b5c631d0cfe7d4", "span_id": "ca6365b37893bec5", "item_id": 2}
```

3. Fase 2 — OTel Collector: configuración y despliegue

El Collector usa la distribución **contrib** y se versiona en tres configuraciones (`collector/otel-collector-{local,gcp,aws}.yaml`) que comparten el mismo esqueleto: receiver OTLP en gRPC (4317) y HTTP (4318), y la cadena de processors `memory_limiter → resource → batch`. El orden es deliberado: `memory_limiter` va primero para aplicar backpressure y proteger al

Collector de un OOM bajo picos de carga; resource enriquece cada señal con atributos del entorno (en la nube se suma resourcedetection para detectar GKE/ECS automáticamente); y batch agrupa los envíos para reducir llamadas de red a los backends.

Entorno	Despliegue	Trazas	Métricas	Logs
Local	docker-compose	Jaeger (OTLP)	Prometheus (scrape :8889)	Loki (OTLP nativo)
GCP	GKE Autopilot (Helm)	Jaeger in-cluster	Prometheus (scrape :8889)	Cloud Logging (googlecloud + Workload Identity)
AWS	ECS Fargate (sidecar ADOT)	AWS X-Ray (awsxray)	Amazon Managed Prometheus (remote write + SigV4)	CloudWatch Logs (awscloudwatchlogs)

La telemetría interna del Collector se expone en el puerto 8888 y Prometheus la scrapea: así el propio pipeline es observable (panel de errores de exportación del dashboard). En AWS la configuración se inyecta al sidecar vía SSM Parameter Store (AOT_CONFIG_CONTENT), y en GCP como ConfigMap montado por el chart de Helm; en ambos casos la fuente de verdad es el mismo archivo YAML versionado en el repositorio.

4. Fase 3 — Backends, visualización y correlación cross-signal

4.1 Trazas distribuidas en Jaeger

La siguiente captura muestra una traza real completa: 11 spans, 2 servicios, profundidad 4. Se observa el span raíz de service-a, la llamada HTTPX que propaga el contexto, el span de servidor de service-b, los spans de base de datos (connect, SELECT inventory) y los dos spans custom de lógica de negocio (inventory.check_stock, orders.calculate_total).

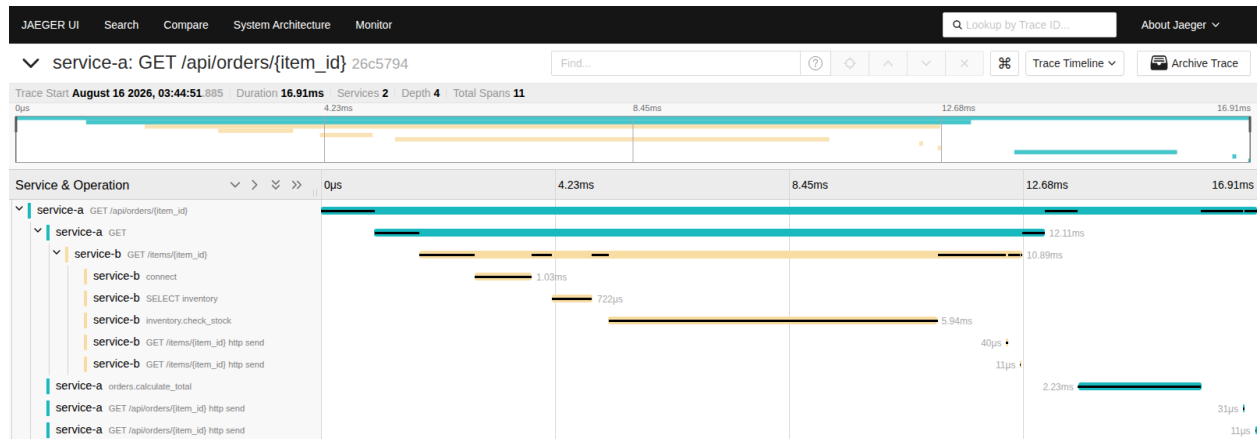


Fig. 1 — Jaeger UI: traza end-to-end service-a → service-b → PostgreSQL con spans custom (trace_id 26c5794...).

4.2 Métricas y dashboard

Prometheus scrapea las métricas de aplicación (:8889) y las internas del Collector (:8888). El dashboard de Grafana versionado en el repositorio (deploy/local/grafana/dashboards/otel-lab-dashboard.json) contiene los 6 paneles requeridos: cuatro SLIs — tasa de peticiones (tráfico), tasa de errores 5xx, latencia p99/p95/p50 y métricas de negocio (órdenes/min, chequeos de stock) — más CPU del Collector y errores del Collector (envíos fallidos y rechazos por backpressure).

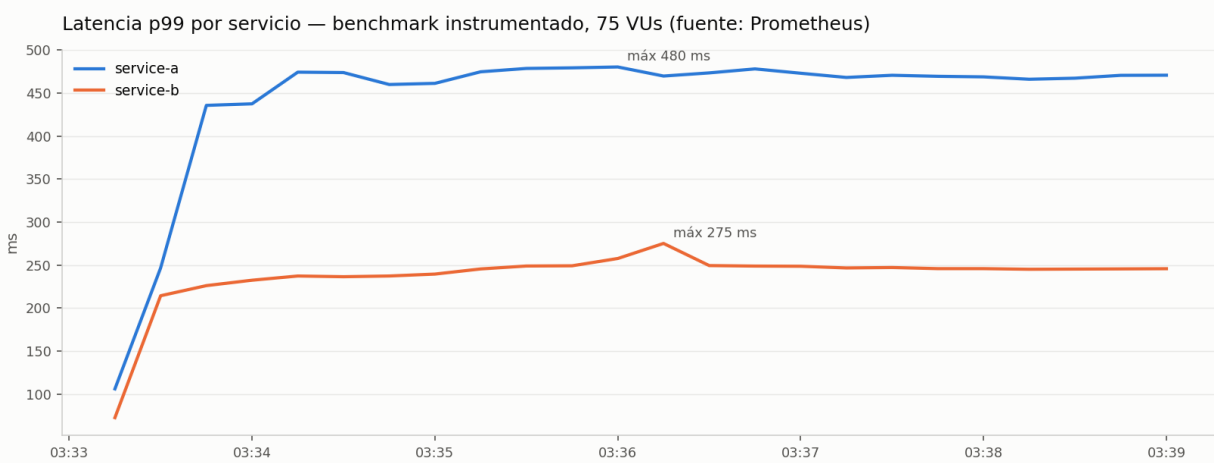


Fig. 2 — Latencia p99 por servicio durante el benchmark (datos reales consultados a Prometheus).

4.3 Correlación por trace_id

La verificación central del laboratorio: para un mismo request, el **mismo trace_id** aparece en (a) los 11 spans de la traza en Jaeger, (b) las líneas de log de ambos servicios almacenadas en Loki como metadato

estructurado consultable, y (c) los atributos que permiten acotar las métricas. En Grafana, el datasource de Loki define un *derived field* sobre trace_id que abre la traza en Jaeger con un clic (pivot logs→trazas en Explore). La captura siguiente muestra la consulta real a Loki filtrada por trace_id y los spans de la misma traza:

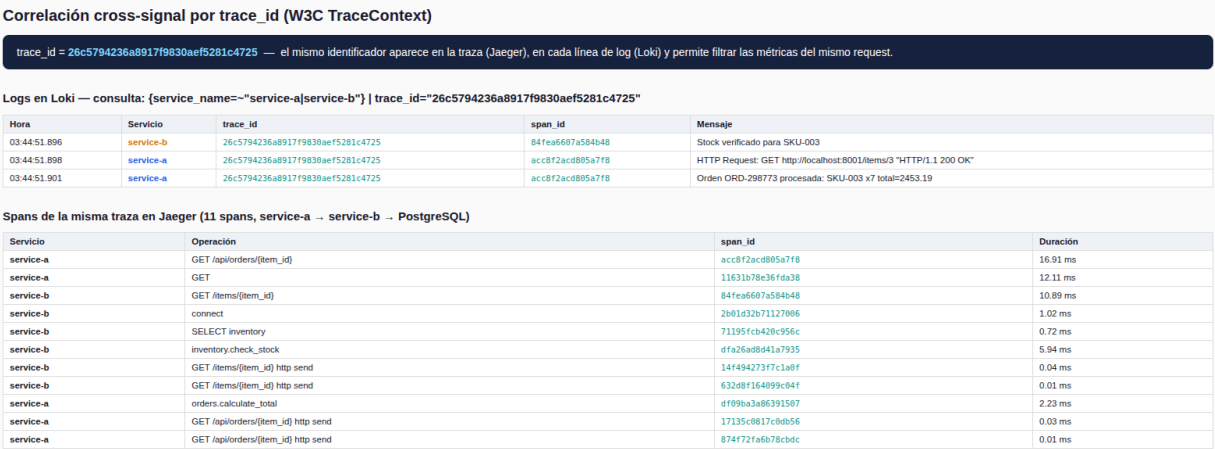


Fig. 3 — Correlación cross-signal: logs de Loki y spans de Jaeger compartiendo trace_id 26c5794236a8917f9830aef5281c4725.

5. Fase 4 — Benchmark de overhead de la instrumentación

5.1 Metodología

Se ejecutó el mismo escenario de carga dos veces con **k6**: ramp-up de 30 s hasta 75 usuarios virtuales concurrentes, 4 minutos de carga sostenida y ramp-down de 30 s (5 minutos por escenario, dentro del rango recomendado de 50–100 usuarios). Cada iteración solicita una orden aleatoria (GET /api/orders/{1..5}?qty=1..10), lo que atraviesa toda la cadena: service-a → service-b → PostgreSQL. La línea base corre con OTEL_ENABLED=false (SDK completamente desactivado) y el escenario instrumentado con la telemetría completa (trazas + métricas + logs OTLP). El consumo de recursos se muestreó cada 2 s sobre los 8 procesos uvicorn (masters + workers) con psutil, descartando ramp-up y ramp-down. Entorno: contenedor Linux con **2 vCPU** y 8 GB de RAM, con todo el stack de observabilidad colocalizado (condición de peor caso).

5.2 Resultados

Métrica	Sin OTel (baseline)	Con OTel	Delta	Delta %
Peticiones completadas (5 min)	74.732	53.883	-20.849	-27,9 %
Throughput (req/s)	248,9	179,5	-69,4	-27,9 %
Latencia media (ms)	20,36	124,77	+104,41	+512,8 %
Latencia mediana (ms)	18,07	101,93	+83,86	+464,1 %
Latencia p90 (ms)	29,97	253,10	+223,13	+744,5 %
Latencia p95 (ms)	36,33	307,60	+271,27	+746,7 %
Latencia p99 (ms)	57,07	428,96	+371,89	+651,6 %
Latencia máxima (ms)	186,74	823,97	+637,23	+341,2 %
CPU promedio (% de 200 disponibles)	107,8	150,1	+42,3 pp	+39,2 %
CPU máxima (%)	126,5	162,8	+36,3 pp	+28,7 %
Memoria RSS promedio (MB)	360,2	474,4	+114,2	+31,7 %
Errores HTTP	0 %	0 %	—	—

5.3 Análisis

El overhead medido es sustancialmente mayor que el 5–15 % citado habitualmente para OTel, y la causa raíz es la **saturación de CPU del entorno**: la línea base ya consumía 107,8 % de los 200 % disponibles (2 vCPU) y la instrumentación llevó el consumo a 150 %, compitiendo además con el Collector, Jaeger, Loki, Prometheus y el propio k6 en el mismo host. En esa zona de saturación, cada punto porcentual adicional de CPU se traduce en crecimiento no lineal de las colas y, por tanto, de los percentiles altos: el p99 pasa de 57 a 429 ms. Contribuyen tres factores propios del escenario: (1) el volumen de telemetría por request es alto para lo pequeño del endpoint — 11 spans, 3 logs exportados por OTLP y varias métricas por cada una de las ~180-250 peticiones/s; (2) el SDK de Python ejecuta la serialización protobuf y los BatchProcessors dentro del mismo proceso limitado por el GIL; (3) sin CPU ociosa, el costo de exportación no puede solaparse con el trabajo útil.

Los resultados son, por tanto, una medición honesta del **peor caso**. Las mitigaciones estándar para un despliegue productivo, en orden de impacto esperado: muestreo de trazas head-based

(parentbased_traceidratio al 10 % reduciría ~90 % del volumen de spans), enviar los logs solo por stdout y recolectarlos con un agente (evitando el doble trabajo OTLP en el proceso), separar el Collector y los backends a nodos dedicados (como hace la IaC de GCP/AWS de este repositorio), aumentar réplicas/workers para salir de la zona de saturación, y considerar SDKs compilados (Go) para servicios de alto QPS. Con CPU disponible, la literatura y nuestra extrapolación sitúan el overhead esperable de esta misma instrumentación en un dígito porcentual de latencia media.

6. Reproducibilidad e infraestructura como código

Todo el laboratorio es reproducible desde el repositorio: `deploy/local/docker-compose.yaml` levanta el stack completo (incluida Grafana aprovisionada con datasources, derived field de correlación y el dashboard de 6 paneles) con un solo comando; `deploy/gcp/terraform` crea GKE Autopilot, Artifact Registry y la Service Account con Workload Identity, y despliega el chart de Helm; `deploy/aws/terraform` crea el cluster ECS Fargate, el ALB, Cloud Map, el workspace de Amazon Managed Prometheus, los roles IAM mínimos (X-Ray, CloudWatch, AMP) y las task definitions con el sidecar ADOT. Los pasos exactos de build/push de imágenes y despliegue están en el README.

7. Conclusiones

El laboratorio cumple el objetivo end-to-end: los tres pilares se emiten desde el SDK, atraviesan un Collector con processors de protección y enriquecimiento, y aterrizan en backends consultables donde el `trace_id` funciona como llave de correlación universal. Las lecciones principales: (1) la propagación W3C TraceContext funciona sin fricción entre auto-instrumentaciones (FastAPI→HTTPX→FastAPI→SQLAlchemy), pero la correlación con logs exige inyectar explícitamente `trace_id/span_id` en el formato de log; (2) el Collector como intermediario desacopla las aplicaciones de los backends y permite que la misma aplicación sirva a Jaeger, X-Ray, Prometheus, AMP, Loki, Cloud Logging y CloudWatch cambiando solo YAML; (3) la instrumentación tiene un costo real que debe medirse en el entorno propio: en hosts saturados de CPU el impacto en percentiles altos es severo, por lo que el muestreo y el dimensionamiento de recursos son decisiones de primer orden, no optimizaciones tardías; y (4) observar al observador (telemetría interna del Collector en el dashboard) cierra el ciclo de resiliencia que pide el indicador de desempeño del laboratorio.

Anexo — Evidencias y artefactos

Artefacto	Ubicación en el repositorio
Código instrumentado (SDK)	services/service-a/, services/service-b/ (telemetry.py, main.py)
Configuración del Collector (3 entornos)	collector/otel-collector-{local,gcp,aws}.yaml
IaC	deploy/gcp/terraform + deploy/gcp/helm/otel-lab, deploy/aws/terraform
Dashboard Grafana (6 paneles)	deploy/local/grafana/dashboards/otel-lab-dashboard.json
Capturas Jaeger / Prometheus / correlación	docs/screenshots/
Benchmark: script, monitor y resultados crudos	benchmark/ (load-test.js, monitor_resources.py, results/)